

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240212 123240218
NAMA	:	Muhammad Athalla Bagaskara Muhammad Ridho Hikmatul Maulana
PLUG	:	IF-A
NAMA ASISTEN	:	Khatama Putra & Bintoro

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA 2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Muhammad Athalla Bagaskara	123240212
Muhammad Ridho Hikmatul	123240218
Maulana	

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

Khatama Putra
NIM. 123230053

Bintoro
NIM. 123230059

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Sistem Pendukung Keputusan Pemilihan Pupuk Terbaik untuk Penanaman Tanaman Padi Berbasis Metode SAW. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 24 Mei 2026

Penyusun 1,

Penyusun 2,

Muhammad Athalla Bagaskara
NIM. 123240212

Muhammad Ridho Hikmatul Maulana
NIM. 123240218

DAFTAR ISI

HALAMAN PENGESAHAN	ii
KATA PENGANTAR.....	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir	1
1.3 Manfaat Proyek Akhir	2
BAB II PEMBAHASAN.....	3
2.1 Dasar Teori	3
2.2 Skenario Kasus & Dataset	3
2.3 Pemodelan Sistem Pendukung Keputusan	4
2.4 Perhitungan & Algoritma	5
2.5 Implementasi & Tampilan Sistem	5
BAB III JADWAL Pengerjaan dan Pembagian Tugas	30
3.1 Jadwal Pengerjaan	30
3.2 Pembagian Tugas.....	30
BAB IV KESIMPULAN DAN SARAN.....	31
4.2 Kesimpulan.....	31
4.3 Saran	31
DAFTAR PUSTAKA.....	32

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Tanaman padi (*Oryza sativa*) merupakan komoditas pangan utama yang sangat krusial bagi ketahanan pangan nasional di Indonesia. Untuk mencapai produktivitas padi yang optimal, pengelolaan unsur hara tanah melalui pemupukan yang tepat sasaran menjadi faktor kunci. Namun, di lapangan, petani seringkali mengalami kesulitan dalam menentukan kombinasi pupuk terbaik karena kondisi kimiawi dan fisik tanah (seperti kadar Nitrogen (N), Fosfor (P), Kalium (K), suhu, kelembaban, pH tanah, dan curah hujan) sangat bervariasi antar lahan. Pemupukan yang tidak seimbang, baik kekurangan maupun kelebihan dosis dapat merusak ekosistem tanah, menurunkan kualitas padi, dan meningkatkan biaya produksi secara tidak efisien.

Untuk mengatasi masalah tersebut, diperlukan sebuah sistem komputasi yang mampu menganalisis berbagai kriteria lingkungan dan memberikan rekomendasi kondisi hara/pupuk terbaik secara presisi. Sistem Pendukung Keputusan (SPK) dengan metode Simple Additive Weighting (SAW) merupakan solusi yang sangat tepat. Metode SAW dipilih karena kemampuannya dalam menyelesaikan masalah optimasi multi-atribut secara dinamis dengan pembobotan kriteria yang dapat disesuaikan. Dengan menerapkan metode ini pada dataset parameter pertanian, sistem dapat meranking alternatif kondisi lahan atau komposisi hara terbaik yang paling sesuai dengan kebutuhan penanaman padi, sehingga membantu petani mengoptimalkan dosis pemupukan.

1.2 Tujuan Proyek Akhir

1. Membangun sebuah aplikasi Sistem Pendukung Keputusan (SPK) berbasis web menggunakan framework Streamlit untuk membantu pemilihan kondisi pupuk dan lahan terbaik bagi tanaman padi.
2. Mengimplementasikan algoritma Simple Additive Weighting (SAW) secara presisi untuk meranking alternatif kondisi hara tanah dan lingkungan berdasarkan 7 kriteria fisik-kimia.
3. Menyediakan visualisasi data analitik yang interaktif agar pengguna dapat menganalisis dampak perubahan bobot preferensi kriteria secara real-time.

1.3 Manfaat Proyek Akhir

1. Bagi Petani & Penyuluh Pertanian: Membantu mempermudah pengambilan keputusan dalam mengidentifikasi kondisi tanah yang optimal dan merekomendasikan kebutuhan pemupukan Nitrogen, Fosfor, dan Kalium (NPK) yang tepat untuk tanaman padi.
2. Bagi Efisiensi Pertanian: Mengurangi pemborosan biaya pembelian pupuk kimia dengan beralih ke sistem rekomendasi berbasis data riil tanah, serta mencegah kerusakan tanah akibat pemupukan berlebih.
3. Bagi Produktivitas Nasional: Meningkatkan potensi hasil panen padi melalui penyesuaian kondisi lahan dengan standar hara optimum tanaman padi.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Metode Simple Additive Weighting (SAW) sering dikenal dengan istilah metode penjumlahan terbobot. Konsep dasar metode SAW adalah mencari penjumlahan terbobot dari rating kinerja pada setiap alternatif pada semua kriteria. Metode SAW membutuhkan proses normalisasi matriks keputusan (X) ke suatu skala yang dapat diperbandingkan dengan semua rating alternatif yang ada.

Langkah-langkah penyelesaian metode SAW adalah sebagai berikut:

1. Menentukan kriteria-kriteria yang akan dijadikan acuan dalam pengambilan keputusan, yaitu C_j .
2. Menentukan rating kecocokan setiap alternatif pada setiap kriteria, dan membentuk matriks keputusan X: $X = [x_{ij}]$
3. Melakukan normalisasi matriks keputusan dengan menghitung nilai rating kinerja ternormalisasi.
4. Menghitung nilai preferensi (V_i) untuk setiap alternatif A_i dengan mengalikan bobot kriteria (w_j) dengan rating kinerja ternormalisasi (r_{ij}): $V_i = \sum (w_j * r_{ij})$ Nilai V_i yang lebih besar mengindikasikan bahwa alternatif A_i lebih terpilih (memiliki peringkat lebih tinggi).

2.2 Skenario Kasus & Dataset

Pengambil keputusan dalam kasus ini adalah kelompok tani atau penyuluh pertanian yang ingin mengevaluasi berbagai sampel kondisi tanah untuk budidaya tanaman padi. Tujuannya adalah mengidentifikasi sampel tanah (alternatif) mana yang memiliki ketersediaan unsur hara makro (N, P, K) dan kondisi lingkungan (suhu, kelembaban, pH, curah hujan) yang paling mendekati profil ideal penanaman padi. Dengan mengetahui alternatif terbaik, petani dapat menentukan lahan mana yang paling produktif atau melakukan intervensi pupuk tertentu untuk menyamakan profil tanah dengan alternatif terbaik tersebut.

Dataset yang digunakan adalah Crop Recommendation Dataset yang bersumber dari platform Kaggle (diunggah oleh atharvaingle). Dataset ini memiliki total 2.200 baris data yang mencakup 22 jenis tanaman (masing-masing 100 data). Untuk analisis khusus budidaya padi, dataset difilter secara spesifik untuk label 'rice' (Padi), sehingga menghasilkan 100 baris alternatif yang dievaluasi.

Dataset ini memiliki 7 kolom kriteria numerik:

1. N (Nitrogen): Kandungan nitrogen dalam tanah (mg/kg).
2. P (Phosphorus): Kandungan fosfor dalam tanah (mg/kg).
3. K (Potassium): Kandungan kalium dalam tanah (mg/kg).
4. temperature: Suhu udara (C).
5. humidity: Kelembaban relatif udara (%).
6. ph: Nilai pH keasaman tanah (0-14).

7. rainfall: Curah hujan rata-rata (mm).

2.3 Pemodelan Sistem Pendukung Keputusan

Sistem pendukung keputusan ini dimodelkan dengan mengidentifikasi alternatif dan kriteria penilaian sebagai berikut:

Alternatif (A_i): 100 sampel lahan padi dari dataset (dataset_pupuk.csv).

Kriteria (C_j):

1. C_1 : Kandungan Nitrogen (N) - Tipe: Benefit.
2. C_2 : Kandungan Fosfor (P) - Tipe: Benefit.
3. C_3 : Kandungan Kalium (K) - Tipe: Benefit.
4. C_4 : Suhu Lingkungan (Temperature) - Tipe: Benefit.
5. C_5 : Kelembaban Udara (Humidity) - Tipe: Benefit.
6. C_6 : pH Tanah (ph) - Tipe: Benefit.
7. C_7 : Curah Hujan (Rainfall) - Tipe: Benefit.

Catatan: Semua kriteria dimodelkan sebagai kriteria Benefit karena ketersediaan hara makro (N, P, K) yang tinggi, kelembaban, pH, suhu, dan curah hujan dalam rentang tumbuh padi berkorelasi positif dengan potensi produktivitas tanaman dalam kerangka optimasi komparatif dataset ini.

Skala Pembobotan:

Bobot kriteria bersifat dinamis (diatur oleh pengguna melalui slider Streamlit).

Untuk analisis awal, digunakan bobot preferensi kelompok sebagai berikut:

- w_1 (Nitrogen) = 20 (20%)
- w_2 (Fosfor) = 15 (15%)
- w_3 (Kalium) = 15 (15%)
- w_4 (Suhu) = 15 (15%)
- w_5 (Kelembaban) = 15 (15%)
- w_6 (pH) = 10 (10%)
- w_7 (Curah Hujan) = 10 (10%)

Total bobot mentah = 100, sehingga bobot ternormalisasi (W) = {0.2, 0.15, 0.15, 0.15, 0.15, 0.1, 0.1}.

Untuk mendemonstrasikan algoritma secara transparan, berikut adalah simulasi perhitungan manual metode SAW menggunakan 3 data sampel padi nyata dari dataset:

1. Alternatif 1 (A_1) (Baris 0): N = 90, P = 42, K = 43, Temp = 20.88, Hum = 82.00, pH = 6.50, Rain = 202.94
2. Alternatif 2 (A_2) (Baris 1): N = 85, P = 58, K = 41, Temp = 21.77, Hum = 80.32, pH = 7.04, Rain = 226.66
3. Alternatif 3 (A_3) (Baris 2): N = 60, P = 55, K = 44, Temp = 23.00, Hum = 82.32, pH = 7.84, Rain = 263.96

2.4 Perhitungan & Algoritma

Alur komputasi metode SAW dalam program SAW_Pupuk.py ditulis menggunakan bahasa Python dengan library Pandas dan NumPy. Alur algoritmanya dirancang sebagai berikut:

1. Inisialisasi Data: Mengambil data dari dataset_pupuk.csv, lalu membersihkan nama kolom dari spasi menggunakan `df.columns = [c.strip() for c in df.columns]`.
2. Pembacaan Parameter & Bobot: Membaca nilai slider bobot kriteria (w_N , w_P , w_K , w_{temp} , $w_{humidity}$, w_{ph} , $w_{rainfall}$) dari sidebar.
3. Normalisasi Bobot: Menghitung bobot ternormalisasi menggunakan fungsi `get_bobot()` agar total penjumlahan bobot sama dengan 1.0.
4. Penerapan Filter: Menyaring data berdasarkan jenis tanaman yang dipilih oleh user (misalnya membatasi hanya pada tanaman padi 'rice').
5. Perhitungan Normalisasi Matriks (R): Menghitung matriks normalisasi R dengan membagi nilai kriteria dengan nilai maksimum kriteria tersebut di dataset (fungsi `normalisasi_saw()`).
6. Perhitungan Preferensi (V): Melakukan perkalian elemen-per-elemen antara baris matriks normalisasi R dengan vektor bobot W, dan menjumlahkannya secara horizontal untuk setiap alternatif (fungsi `hitung_saw()`).
7. Perangkingan: Menyortir alternatif berdasarkan Nilai Preferensi (V) secara descending (terbesar ke terkecil), lalu menambahkan kolom peringkat untuk visualisasi tabel hasil akhir.

2.5 Implementasi & Tampilan Sistem

Berikut adalah source code lengkap untuk aplikasi SPK Pemilihan Pupuk Terbaik berbasis Streamlit (SAW_Pupuk.py):

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns

st.set_page_config(
    page_title="SPK Pupuk Padi - SAW",
    page_icon="🌱",
    layout="wide",
    initial_sidebar_state="expanded",
)

st.markdown("""
<style>
```

```

                                                                    @import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&
display=swap');
html, body, [class*="css"] { font-family: 'Poppins', sans-serif; }
.main { background-color: #f0f7ee; }
.card {
    background: white;
    border-radius: 12px;
    padding: 20px 24px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.08);
    margin-bottom: 16px;
}
.metric-card {
    background: linear-gradient(135deg, #2d6a4f, #52b788);
    border-radius: 12px;
    padding: 18px 22px;
    color: white;
    text-align: center;
    box-shadow: 0 4px 12px rgba(45,106,79,0.3);
}
.metric-card h2 { margin: 0; font-size: 2rem; font-weight: 700; }
.metric-card p { margin: 0; font-size: 0.85rem; opacity: 0.85; }
.title-header {
    background: linear-gradient(135deg, #1b4332, #2d6a4f);
    padding: 28px 32px;
    border-radius: 14px;
    color: white;
    margin-bottom: 24px;
}
.title-header h1 { margin: 0; font-size: 1.7rem; font-weight: 700; }
.title-header p { margin: 4px 0 0 0; opacity: 0.85; font-size:
0.9rem; }
.formula-box {
    background: #e8f5e9;
    border-left: 4px solid #2d6a4f;
    border-radius: 0 8px 8px 0;
    padding: 14px 18px;
    font-family: monospace;
    font-size: 0.95rem;
    color: #1b4332;
    margin: 10px 0;
}
div[data-testid="stSidebar"] {
    background: linear-gradient(180deg, #1b4332 0%, #2d6a4f 100%);
}
div[data-testid="stSidebar"] * { color: white !important; }

```

```

    div[data-testid="stSidebar"] .stSlider > div > div > div { background:
#52b788 !important; }
    div[data-testid="stSidebar"] label { color: #d8f3dc !important; font-
size: 0.85rem; }
    div[data-testid="stSidebar"] h2, div[data-testid="stSidebar"] h3 {
color: #d8f3dc !important; }
    .stButton > button {
        background: linear-gradient(135deg, #2d6a4f, #40916c);
        color: white;
        border: none;
        border-radius: 8px;
        padding: 10px 28px;
        font-weight: 600;
        font-size: 1rem;
        transition: all 0.2s;
    }
    .stButton > button:hover {
        background: linear-gradient(135deg, #1b4332, #2d6a4f);
        transform: translateY(-1px);
        box-shadow: 0 4px 12px rgba(45,106,79,0.4);
    }
    .highlight-rice {
        background: #d8f3dc;
        border: 1.5px solid #52b788;
        border-radius: 6px;
        padding: 6px 12px;
        font-weight: 600;
        color: #1b4332;
    }
    .info-box {
        background: #e8f5e9;
        border: 1px solid #52b788;
        border-radius: 8px;
        padding: 12px 16px;
        margin-bottom: 12px;
        font-size: 0.9rem;
        color: #1b4332;
    }
}
</style>
""", unsafe_allow_html=True)

# LOAD DATASET
import os
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

@st.cache_data

```

```

def load_data():
    csv_path = os.path.join(SCRIPT_DIR, "dataset_pupuk.csv")
    df = pd.read_csv(csv_path)
    df.columns = [c.strip() for c in df.columns]
    return df

df_all = load_data()

# SIDEBAR - NAVIGASI & BOBOT
# -----
with st.sidebar:
    st.markdown("## 🌾 SPK Pupuk Padi")
    st.markdown("---")
    halaman = st.selectbox(
        "📌 Pilih Halaman",
        ["🏠 Beranda", "📊 Dataset", "⚙️ Hitung SPK-SAW", "📈 Visualisasi Data", "👥 Profil Kelompok"]
    )
    st.markdown("---")
    st.markdown("### ⚖️ Bobot Kriteria")
    st.markdown("*Atur bobot tiap kriteria – berlaku di semua halaman*")

    w_N = st.slider("🌿 Nitrogen (N)", 0, 100, 20, key="wN")
    w_P = st.slider("🟡 Fosfor (P)", 0, 100, 15, key="wP")
    w_K = st.slider("🟢 Kalium (K)", 0, 100, 15, key="wK")
    w_temp = st.slider("🔥 Suhu (Temperature)", 0, 100, 15, key="wT")
    w_hum = st.slider("💧 Kelembaban (Humidity)", 0, 100, 15, key="wH")
    w_ph = st.slider("🧪 pH Tanah", 0, 100, 10, key="wPH")
    w_rain = st.slider("🌧️ Curah Hujan (Rainfall)", 0, 100, 10, key="wR")

    st.markdown("---")
    st.markdown("### 🌱 Filter Tanaman")
    crop_filter = st.selectbox(
        "Fokus tanaman:",
        ["Semua Tanaman", "rice (Padi)"] +
        [c for c in sorted(df_all['label'].unique()) if c != 'rice']
    )
    st.markdown("---")
    st.markdown("### 📊 Jumlah Hasil Ranking")
    top_n = st.number_input("Tampilkan Top-N:", min_value=5, max_value=200, value=20, step=5)

```

```

# HELPER FUNGSI SAW
# -----
KRITERIA = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
TIPE = ['benefit', 'benefit', 'benefit', 'benefit', 'benefit', 'benefit', 'benefit']

LABEL_MAP = {
    'N': 'Nitrogen (N)',
    'P': 'Fosfor (P)',
    'K': 'Kalium (K)',
    'temperature': 'Suhu',
    'humidity': 'Kelembaban',
    'ph': 'pH Tanah',
    'rainfall': 'Curah Hujan'
}

WARNA_HIJAU = ['#1b4332', '#2d6a4f', '#40916c', '#52b788', '#74c69d', '#95d5b2', '#b7e4c7']

def get_bobot(w_N, w_P, w_K, w_temp, w_hum, w_ph, w_rain):
    raw = np.array([w_N, w_P, w_K, w_temp, w_hum, w_ph, w_rain],
dtype=float)
    total = raw.sum()
    if total == 0:
        return raw
    return raw / total

def normalisasi_saw(df_input):
    df_norm = pd.DataFrame(index=df_input.index)
    for i, col in enumerate(KRITERIA):
        if TIPE[i] == 'benefit':
            max_val = df_input[col].max()
            df_norm[col] = df_input[col] / max_val if max_val != 0 else
0
        else:
            min_val = df_input[col].min()
            df_norm[col] = min_val / df_input[col].replace(0, np.nan)
    return df_norm

def hitung_saw(df_input, bobot):
    df_norm = normalisasi_saw(df_input)
    nilai = (df_norm[KRITERIA] * bobot).sum(axis=1)
    return df_norm, nilai

def get_filtered_df():
    """Kembalikan dataframe sesuai filter tanaman di sidebar."""

```

```

if crop_filter == "Semua Tanaman":
    return df_all.copy()
else:
    crop_name = crop_filter.split(" ")[0]
    return df_all[df_all['label'] == crop_name].copy()

def get_saw_result(df_input):
    """Hitung SAW dengan bobot aktif dari sidebar."""
    bobot = get_bobot(w_N, w_P, w_K, w_temp, w_hum, w_ph, w_rain)
    df_w = df_input.reset_index(drop=True)
    df_norm, nilai = hitung_saw(df_w[KRITERIA], bobot)
    df_hasil = df_w.copy()
    df_hasil['Nilai V'] = nilai.values
    df_hasil = df_hasil.sort_values('Nilai V',
ascending=False).reset_index(drop=True)
    df_hasil.insert(0, 'Peringkat', range(1, len(df_hasil)+1))
    return df_hasil, bobot

# HALAMAN 1 - BERANDA
# -----
if halaman == "🏠 Beranda":
    st.markdown("""
<div class='title-header'>
    <h1>🌱 Sistem Pendukung Keputusan Pemilihan Pupuk Terbaik</h1>
    <p>Metode SAW (Simple Additive Weighting) – Praktikum SCPK
2025/2026</p>
</div>
""", unsafe_allow_html=True)

    col1, col2, col3, col4 = st.columns(4)
    with col1:
        st.markdown("<div class='metric-card'><h2>2.200</h2><p>Total
Data Sampel</p></div>", unsafe_allow_html=True)
    with col2:
        st.markdown("<div class='metric-card'><h2>7</h2><p>Kriteria
Penilaian</p></div>", unsafe_allow_html=True)
    with col3:
        st.markdown("<div class='metric-card'><h2>22</h2><p>Jenis
Tanaman</p></div>", unsafe_allow_html=True)
    with col4:
        st.markdown("<div class='metric-card'><h2>SAW</h2><p>Metode
SPK</p></div>", unsafe_allow_html=True)

    st.markdown("---")
    col_a, col_b = st.columns([1.2, 1])
    with col_a:

```

```

st.markdown("<div class='card'>", unsafe_allow_html=True)
st.subheader("📄 Tentang Sistem")
st.markdown("""
Sistem ini merupakan Sistem Pendukung Keputusan (SPK) berbasis
metode
    SAW (Simple Additive Weighting) untuk menganalisis dan
meranking kondisi
    lahan yang paling optimal untuk penanaman Tanaman Padi 🌾.

Sumber Dataset:
- 📦 Kaggle – Crop Recommendation Dataset (atharvaingle)
- ✅ 2.200 baris data, 7 kriteria, 22 jenis tanaman
- 📍 Dikumpulkan dari stasiun penelitian pertanian

Kriteria yang digunakan:
| Kriteria | Satuan | Tipe |
|---|---|---|
| Nitrogen (N) | mg/kg | Benefit |
| Fosfor (P) | mg/kg | Benefit |
| Kalium (K) | mg/kg | Benefit |
| Suhu | °C | Benefit |
| Kelembaban | % | Benefit |
| pH Tanah | – | Benefit |
| Curah Hujan | mm | Benefit |
""")
st.markdown("</div>", unsafe_allow_html=True)

with col_b:
    st.markdown("<div class='card'>", unsafe_allow_html=True)
    st.subheader("📊 Rumus Metode SAW")
    st.markdown("Langkah 1 – Normalisasi Matriks:")
    st.markdown("""
<div class='formula-box'>
Benefit :  $r_{ij} = x_{ij} / \max(x_j)$ <br>
Cost :  $r_{ij} = \min(x_j) / x_{ij}$ 
</div>
""", unsafe_allow_html=True)
    st.markdown("Langkah 2 – Nilai Preferensi:")
    st.markdown("""
<div class='formula-box'>
 $V_i = \sum (w_j \times r_{ij})$ 
</div>
""", unsafe_allow_html=True)
    st.markdown("""
Keterangan:

```

```

- `r_ij` = nilai normalisasi kriteria j pada alternatif i
- `w_j` = bobot kriteria j ( $\sum w_j = 1$ )
- `V_i` = nilai preferensi akhir alternatif i
- Alternatif dengan  $\sum V_i$  terbesar = Peringkat 1**
""")
st.markdown("</div>", unsafe_allow_html=True)

# HALAMAN 2 - DATASET
# -----
elif halaman == "📊 Dataset":
    st.markdown("<div class='title-header'><h1>📊 Dataset Crop Recommendation</h1><p>Sumber: Kaggle – atharvaingle | 2.200 baris, 8 kolom</p></div>", unsafe_allow_html=True)

    col1, col2 = st.columns([1,1])
    with col1:
        st.markdown("<div class='card'>", unsafe_allow_html=True)
        st.markdown("#### 🔍 Info Dataset")
        st.markdown(f"- **Total baris:** {len(df_all):,}")
        st.markdown(f"- **Total kolom:** {df_all.shape[1]}")
        st.markdown(f"- **Missing values:** {df_all.isnull().sum().sum()}")
        st.markdown(f"- **Duplikat:** {df_all.duplicated().sum()}")
        rice_count = len(df_all[df_all['label']=='rice'])
        st.markdown(f"- **Baris Rice/Padi:** <span class='highlight-rice'>{rice_count} baris</span>", unsafe_allow_html=True)
        st.markdown("</div>", unsafe_allow_html=True)
    with col2:
        st.markdown("<div class='card'>", unsafe_allow_html=True)
        st.markdown("#### 🚩 Filter Tampilan Dataset")
        show_crop = st.selectbox("Pilih tanaman:", ["Semua"] + sorted(df_all['label'].unique().tolist()))
        if show_crop != "Semua":
            df_show = df_all[df_all['label'] == show_crop]
        else:
            df_show = df_all
        st.markdown(f"Menampilkan **{len(df_show):,}** baris")
        st.markdown("</div>", unsafe_allow_html=True)

    st.markdown("#### 📄 Tabel Data Mentah")
    st.dataframe(df_show.reset_index(drop=True),
use_container_width=True, height=400)
    st.markdown("----")
    st.markdown("#### 📈 Statistik Deskriptif")
    st.dataframe(df_show.describe().round(3), use_container_width=True)

```



```

# HALAMAN 3 - HITUNG SPK SAW
# -----
elif halaman == "⚙️ Hitung SPK-SAW":
    st.markdown("<div class='title-header'><h1>⚙️ Perhitungan SPK -  
Metode SAW</h1><p>Simple Additive Weighting – Atur bobot di sidebar  
kiri</p></div>", unsafe_allow_html=True)

    df_hitung = get_filtered_df()
    df_hitung = df_hitung.reset_index(drop=True)
    df_hitung.index = df_hitung.index + 1
    df_hitung.index.name = "No"

    bobot = get_bobot(w_N, w_P, w_K, w_temp, w_hum, w_ph, w_rain)

    st.markdown("----")
    st.markdown("### 📊 Bobot Kriteria yang Digunakan")
    df_bobot = pd.DataFrame({
        "Kriteria": [LABEL_MAP[k] for k in KRITERIA],
        "Bobot (Input)": [w_N, w_P, w_K, w_temp, w_hum, w_ph, w_rain],
        "Bobot (Ternormalisasi)": [round(b, 4) for b in bobot],
        "Tipe": TIPE
    })
    st.dataframe(df_bobot, use_container_width=True, hide_index=True)
    st.markdown("----")

    col_btn, _ = st.columns([1, 3])
    with col_btn:
        hitung = st.button("🚀 Hitung SPK-SAW Sekarang",
use_container_width=True)

    if hitung:
        with st.spinner("Menghitung... 🔄"):
            df_norm, nilai_preferensi = hitung_saw(df_hitung[KRITERIA],
bobot)

            df_hasil = df_hitung.copy()
            df_hasil['Nilai Preferensi (V)'] = nilai_preferensi.values
            df_hasil = df_hasil.sort_values('Nilai Preferensi (V)',
ascending=False)
            df_hasil.insert(0, 'Peringkat', range(1, len(df_hasil)+1))

            st.markdown("----")
            st.markdown("### 📌 Langkah 1 – Matriks Keputusan (Data  
Asli)")

```

```

        st.markdown(f"*Menampilkan {min(10, len(df_hitung))} baris pertama*")

        st.dataframe(df_hitung[KRITERIA].head(10).round(4),
use_container_width=True)

        st.markdown("----")
        st.markdown("### 🚀 Langkah 2 – Matriks Normalisasi (rij)")
        st.markdown("""<div class='formula-box'>Benefit:  $r_{ij} = x_{ij} / \max(x_j)$ </div>""", unsafe_allow_html=True)
        df_norm_show = df_norm.copy()
        df_norm_show.columns = [LABEL_MAP[c] for c in KRITERIA]
        st.dataframe(df_norm_show.head(10).round(4),
use_container_width=True)

        st.markdown("----")
        st.markdown("### 🏆 Langkah 3 – Nilai Preferensi ( $V_i = \sum w_j \times r_{ij}$ )")
        st.markdown("""<div class='formula-box'> $V_i = w_N \times r_N + w_P \times r_P + w_K \times r_K + w_T \times r_T + w_H \times r_H + w_{pH} \times r_{pH} + w_R \times r_R$ </div>""",
unsafe_allow_html=True)

        st.markdown("----")
        st.markdown(f"### 🏆 Hasil Perangkingan SAW (Top {top_n})")
        df_top = df_hasil.head(int(top_n)).copy()
        df_top_show = df_top[['Peringkat', 'label', 'N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall', 'Nilai Preferensi (V)']].copy()
        df_top_show.columns = ['Peringkat', 'Tanaman', 'N', 'P', 'K', 'Suhu', 'Kelembaban', 'pH', 'Curah Hujan', 'Nilai V']
        df_top_show['Nilai V'] = df_top_show['Nilai V'].round(4)

        def warna_peringkat(row):
            if row['Peringkat'] == 1:
                return ['background-color: #FFF9C4; font-weight:bold']*len(row)
            elif row['Peringkat'] == 2:
                return ['background-color: #F5F5F5; font-weight:bold']*len(row)
            elif row['Peringkat'] == 3:
                return ['background-color: #FFF0E0; font-weight:bold']*len(row)
            return ['']*len(row)

        st.dataframe(
            df_top_show.style.apply(warna_peringkat, axis=1),

```

```

        use_container_width=True, height=450, hide_index=True
    )

    st.markdown("---")
    st.markdown("### 🏆 Top 3 Terbaik")
    medals = ["🥇", "🥈", "🥉"]
    col3 = st.columns(3)
    for idx in range(min(3, len(df_hasil))):
        row = df_hasil.iloc[idx]
        with col3[idx]:
            st.markdown(f"""
                <div class='card'>
                    <h2 style='margin:0;font-size:2rem'>{medals[idx]}</h2>
                    <h3 style='color:#2d6a4f;margin:4px 0'>{row['label'].upper()}</h3>
                    <p style='margin:0;font-size:0.9rem'>N={row['N']} | P={row['P']} | K={row['K']}</p>
                    <p style='margin:4px 0;font-size:0.9rem'>Suhu={row['temperature']:.1f}°C | pH={row['ph']:.2f}</p>
                    <p style='font-weight:700;color:#1b4332;font-size:1.1rem'>V = {row['Nilai Preferensi (V)']:.4f}</p>
                </div>
                """, unsafe_allow_html=True)

    st.success(f"✅ Perhitungan SAW selesai! {len(df_hitung):,} alternatif diranking berdasarkan 7 kriteria.")

# HALAMAN 4 - VISUALISASI (DINAMIS)
# -----
elif halaman == "📊 Visualisasi Data":
    st.markdown(
        """<div class='title-header'><h1>📊 Visualisasi & Analitik Data</h1>
        <p>Semua grafik otomatis berubah sesuai Filter Tanaman & Bobot Kriteria di sidebar</p></div>""",
        unsafe_allow_html=True
    )

    # Ambil data sesuai filter sidebar
    df_vis = get_filtered_df().reset_index(drop=True)
    bobot = get_bobot(w_N, w_P, w_K, w_temp, w_hum, w_ph, w_rain)

    # Hitung nilai SAW untuk data yang sudah difilter
    df_norm_vis, nilai_vis = hitung_saw(df_vis[KRITERIA], bobot)
    df_vis['Nilai V'] = nilai_vis.values

```

```
# Buat label kategori rice vs lain
df_vis['Kategori'] = df_vis['label'].apply(lambda x: 'Rice (Padi)'
if x == 'rice' else 'Tanaman Lain')

# Info konteks dinamis
label_aktif = crop_filter
jumlah_data = len(df_vis)
bobot_pct = [round(b*100, 1) for b in bobot]
bobot_dominan = LABEL_MAP[KRITERIA[int(np.argmax(bobot))]] if
bobot.sum() > 0 else "-"

st.markdown(f"""
<div class='info-box'>
    📌 <b>Filter aktif:</b> {label_aktif} &nbsp;|&nbsp;
    📦 <b>Data:</b> {jumlah_data:,} baris &nbsp;|&nbsp;
    🏆 <b>Bobot terbesar:</b> {bobot_dominan} ({max(bobot_pct):.1f}%)
    &nbsp;|&nbsp;
    ⚖️ <b>Bobot (%):</b> N={bobot_pct[0]} P={bobot_pct[1]}
    K={bobot_pct[2]}
    Suhu={bobot_pct[3]} Lembab={bobot_pct[4]} pH={bobot_pct[5]}
    Hujan={bobot_pct[6]}
</div>
""", unsafe_allow_html=True)

# GRAFIK 1 – DISTRIBUSI NILAI SAW (DINAMIS)
# =====
st.markdown("### 🇮🇩 Grafik 1 – Distribusi Nilai Preferensi SAW")
st.caption("💡 *Grafik ini berubah saat kamu menggeser slider bobot
atau mengganti filter tanaman*")

fig1, axes1 = plt.subplots(1, 2, figsize=(14, 5))

# Kiri: Histogram nilai V per tanaman (top 10 crop)
top_crops = df_vis.groupby('label')['Nilai
V'].mean().nlargest(10).index.tolist()
df_top10 = df_vis[df_vis['label'].isin(top_crops)]
colors_map = {crop: WARNA_HIJAU[i % len(WARNA_HIJAU)] for i, crop in
enumerate(top_crops)}

for crop in top_crops:
    subset = df_top10[df_top10['label'] == crop]['Nilai V']
    axes1[0].hist(subset, bins=15, alpha=0.6,
color=colors_map[crop], label=crop,
edgecolor='white')
```

```

axes1[0].set_xlabel("Nilai Preferensi (V)", fontsize=11)
axes1[0].set_ylabel("Frekuensi", fontsize=11)
axes1[0].set_title(f"Distribusi Nilai V per Tanaman\n(Top-10 rata-
rata tertinggi)", fontweight='bold')
axes1[0].legend(fontsize=7, ncol=2)
axes1[0].set_facecolor('#f8fffe')
axes1[0].grid(axis='y', linestyle='--', alpha=0.4)

# Kanan: Bar rata-rata nilai V per tanaman, diurutkan
mean_v = df_vis.groupby('label')['Nilai
V'].mean().sort_values(ascending=False).head(15)
bar_colors = ['#FFD700' if c == 'rice' else WARNA_HIJAU[i %
len(WARNA_HIJAU)]
               for i, c in enumerate(mean_v.index)]
bars = axes1[1].barh(mean_v.index[::-1], mean_v.values[::-1],
                    color=bar_colors[::-1], edgecolor='white',
height=0.65)
axes1[1].set_xlabel("Rata-rata Nilai V", fontsize=11)
axes1[1].set_title(f"Rata-rata Nilai V per Tanaman\n(Bobot:
N={bobot_pct[0]}% P={bobot_pct[1]}% K={bobot_pct[2]}%...)",
                    fontweight='bold')
axes1[1].set_facecolor('#f8fffe')
axes1[1].grid(axis='x', linestyle='--', alpha=0.4)
for bar in bars:
    axes1[1].text(bar.get_width() + 0.002, bar.get_y() +
bar.get_height()/2,
                  f"{bar.get_width():.3f}", va='center', fontsize=8)

# Tambahkan legenda rice
if 'rice' in mean_v.index:
    patch_rice = mpatches.Patch(color='#FFD700', label='Rice (Padi)')
    axes1[1].legend(handles=[patch_rice], loc='lower right')

plt.tight_layout()
st.pyplot(fig1)
plt.close()

st.markdown("---")

# GRAFIK 2 – KONTRIBUSI BOBOT KRITERIA
# =====
st.markdown("### 🇮🇩 Grafik 2 – Kontribusi Bobot Kriteria (Aktif)")
st.caption("💡 *Pie chart & bar ini langsung berubah saat kamu
menggeser slider bobot di sidebar*")

fig2, axes2 = plt.subplots(1, 2, figsize=(14, 5))

```

```

label_kriteria = [LABEL_MAP[k] for k in KRITERIA]
raw_bobot = [w_N, w_P, w_K, w_temp, w_hum, w_ph, w_rain]

# Kiri: Pie chart bobot
if sum(raw_bobot) > 0:
    wedges, texts, autotexts = axes2[0].pie(
        raw_bobot, labels=label_kriteria,
        autopct=lambda p: f'{p:.1f}%' if p > 3 else '',
        colors=WARNA_HIJAU,
        startangle=140, pctdistance=0.75,
        wedgeprops=dict(edgecolor='white', linewidth=1.5)
    )
    for at in autotexts:
        at.set_fontsize(9)
        at.set_color('white')
        at.set_fontweight('bold')
else:
    axes2[0].text(0.5, 0.5, 'Semua bobot = 0', ha='center',
va='center', fontsize=13)
    axes2[0].set_title("Proporsi Bobot Kriteria\n(sesuai slider
sidebar)", fontweight='bold')

# Kanan: Bar chart bobot ternormalisasi
x_pos = range(len(KRITERIA))
bar2 = axes2[1].bar(x_pos, bobot, color=WARNA_HIJAU,
edgecolor='white', linewidth=0.8)
axes2[1].set_xticks(list(x_pos))
axes2[1].set_xticklabels(label_kriteria, rotation=30, ha='right',
fontsize=9)
axes2[1].set_ylabel("Bobot Ternormalisasi", fontsize=11)
axes2[1].set_title("Bobot Kriteria Ternormalisasi\n(jumlah = 1.00)",
fontweight='bold')
axes2[1].set_facecolor('#f8fffe')
axes2[1].grid(axis='y', linestyle='--', alpha=0.4)
for bar, val in zip(bar2, bobot):
    axes2[1].text(bar.get_x() + bar.get_width()/2, bar.get_height()
+ 0.003,
                    f"{val:.3f}", ha='center', va='bottom', fontsize=9,
fontweight='bold')

plt.tight_layout()
st.pyplot(fig2)
plt.close()

st.markdown("---")

```

```

# GRAFIK 3 – BOXPLOT KRITERIA
# =====
st.markdown("### 📊 Grafik 3 – Distribusi Nilai Kriteria per Kelompok")
st.caption("💡 *Boxplot ini berubah sesuai filter tanaman yang dipilih di sidebar*")

fig3, axes3 = plt.subplots(2, 4, figsize=(16, 9))
axes3_flat = axes3.flatten()

for i, col in enumerate(KRITERIA):
    ax = axes3_flat[i]
    if crop_filter == "Semua Tanaman":
        # Semua tanaman: rice vs lain
        df_bp = df_vis[['label', col, 'Kategori']].copy()
        palette = {'Rice (Padi)': '#52b788', 'Tanaman Lain': '#b7e4c7'}
        sns.boxplot(data=df_bp, x='Kategori', y=col, ax=ax,
                    palette=palette, width=0.5, linewidth=1.5)
        ax.set_xlabel("")
    else:
        # Satu tanaman: tampilkan distribusi per label (hanya 1 label)
        df_bp = df_vis[[col]].copy()
        ax.boxplot(df_bp[col].dropna(), vert=True, patch_artist=True,
                  boxprops=dict(facecolor='#52b788',
                                medianprops=dict(color='#FFD700', linewidth=2),
                                whiskerprops=dict(color='#2d6a4f'),
                                capprops=dict(color='#2d6a4f')),
                  color='#1b4332'),
                  medianprops=dict(color='#FFD700', linewidth=2),
                  whiskerprops=dict(color='#2d6a4f'),
                  capprops=dict(color='#2d6a4f'))
        ax.set_xticks([1])
        ax.set_xticklabels([crop_filter.split(" ")[0]])

        ax.set_title(f"{LABEL_MAP[col]}\n(bobot: {bobot_pct[i]:.1f}%)",
                    fontweight='bold', fontsize=9)
        ax.set_ylabel(col, fontsize=9)
        ax.set_facecolor('#f8fffe')
        ax.grid(axis='y', linestyle='--', alpha=0.4)

# Slot ke-8 kosong – tampilkan ringkasan bobot
ax_extra = axes3_flat[7]
ax_extra.axis('off')
bobot_teks = "\n".join([f"{LABEL_MAP[k]}: {bobot_pct[i]:.1f}%" for i, k in enumerate(KRITERIA)])
ax_extra.text(0.1, 0.95, f"📊 Bobot Aktif:\n\n{bobot_teks}",

```

```

        transform=ax_extra.transAxes,
        fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='#d8f3dc',
alpha=0.8))

fig3.suptitle(
    f"Distribusi 7 Kriteria – Filter: {crop_filter}",
    fontsize=13, fontweight='bold'
)
plt.tight_layout()
st.pyplot(fig3)
plt.close()

st.markdown("---")

# GRAFIK 4 – RADAR CHART
# =====
st.markdown("### 📊 Grafik 4 – Radar Chart Profil Kriteria")
st.caption("💡 *Radar chart berubah saat filter tanaman diganti.
Titik merah = profil tanaman terpilih / Rice*")

all_max = df_all[KRITERIA].max()
rice_mean = df_all[df_all['label']=='rice'][KRITERIA].mean()
all_mean = df_all[KRITERIA].mean()
rice_norm = (rice_mean / all_max).values.tolist()
all_norm = (all_mean / all_max).values.tolist()

# Profil tanaman yang difilter
if crop_filter != "Semua Tanaman":
    crop_name = crop_filter.split(" ")[0]
    filter_mean =
df_all[df_all['label']==crop_name][KRITERIA].mean()
    filter_norm = (filter_mean / all_max).values.tolist()
    has_filter = True
else:
    has_filter = False

# Bobot ternormalisasi untuk radar
bobot_norm_radar = (bobot / bobot.max()).tolist() if bobot.max() > 0
else bobot.tolist()

labels_r = [LABEL_MAP[k] for k in KRITERIA]
N_axis = len(labels_r)
angles = [n / float(N_axis) * 2 * np.pi for n in range(N_axis)]
angles += angles[:1]

```



```

fig4, axes4 = plt.subplots(1, 2, figsize=(14, 6),
subplot_kw=dict(polar=True))

# Radar kiri: Profil kriteria tanaman
ax_r1 = axes4[0]
r_all = all_norm + all_norm[:1]
r_rice = rice_norm + rice_norm[:1]
ax_r1.plot(angles, r_all, 'o--', lw=1.5, color='#95d5b2',
label='Rata-rata Semua')
ax_r1.fill(angles, r_all, alpha=0.10, color='#95d5b2')
ax_r1.plot(angles, r_rice, 'o-', lw=2, color='#2d6a4f',
label='Rice (Padi)')
ax_r1.fill(angles, r_rice, alpha=0.25, color='#52b788')
if has_filter:
    r_filt = filter_norm + filter_norm[:1]
    ax_r1.plot(angles, r_filt, 's-', lw=2, color='#e63946',
label=crop_name)
    ax_r1.fill(angles, r_filt, alpha=0.15, color='#e63946')
ax_r1.set_xticks(angles[:-1])
ax_r1.set_xticklabels(labels_r, fontsize=9)
ax_r1.set_title("Profil Kriteria Tanaman\n(Ternormalisasi 0-1)",
fontweight='bold', pad=20)
ax_r1.legend(loc='upper right', bbox_to_anchor=(1.4, 1.15),
fontsize=8)
ax_r1.set_facecolor('#f0f7ee')
ax_r1.grid(color='#c7e9c0', linewidth=0.8)

# Radar kanan: Visualisasi bobot aktif
ax_r2 = axes4[1]
r_bobot = bobot_norm_radar + bobot_norm_radar[:1]
ax_r2.plot(angles, r_bobot, 'o-', lw=2.5, color='#f4a261',
label='Bobot Aktif')
ax_r2.fill(angles, r_bobot, alpha=0.30, color='#f4a261')
ax_r2.set_xticks(angles[:-1])
ax_r2.set_xticklabels([f"{labels_r[i]}\n({bobot_pct[i]}%)" for i in
range(N_axis)], fontsize=8)
ax_r2.set_title("Visualisasi Bobot Kriteria Aktif\n(Ternormalisasi
thd bobot terbesar)",
fontweight='bold', pad=20)
ax_r2.set_facecolor('#fff8f0')
ax_r2.grid(color='#ffe0b2', linewidth=0.8)

plt.tight_layout()
st.pyplot(fig4)
plt.close()

```

```

st.markdown("---")

# GRAFIK 5 – HEATMAP KORELASI
# =====

st.markdown("### 📊 Grafik 5 – Heatmap Korelasi Antar Kriteria")
st.caption("💡 *Korelasi dihitung ulang sesuai data yang sedang difilter*")

fig5, ax5 = plt.subplots(figsize=(9, 7))
corr_data = df_vis[KRITERIA + ['Nilai V']].corr()
mask = np.triu(np.ones_like(corr_data, dtype=bool))
labels_corr = [LABEL_MAP[k] for k in KRITERIA] + ['Nilai V (SAW)']
sns.heatmap(
    corr_data, annot=True, fmt=".2f", cmap='YlGn',
    mask=mask, ax=ax5, linewidths=0.5,
    cbar_kws={'shrink': 0.8},
    xticklabels=labels_corr,
    yticklabels=labels_corr
)
ax5.set_title(
    f"Korelasi Antar Kriteria + Nilai SAW\nFilter: {crop_filter} | {jumlah_data:,} data",
    fontweight='bold', fontsize=12
)
plt.xticks(rotation=35, ha='right')
plt.tight_layout()
st.pyplot(fig5)
plt.close()

st.markdown("---")

# GRAFIK 6 – SCATTER TOP-N SAW
# =====

st.markdown("### 📊 Grafik 6 – Scatter Plot Top Data Berdasarkan Nilai SAW")
st.caption("💡 *Titik-titik emas = top 10% nilai V tertinggi berdasarkan bobot aktif. Berubah tiap kali bobot digeser!*")

fig6, ax6 = plt.subplots(figsize=(11, 6))

threshold_top = df_vis['Nilai V'].quantile(0.90)
df_top_scatter = df_vis[df_vis['Nilai V'] >= threshold_top]
df_bot_scatter = df_vis[df_vis['Nilai V'] < threshold_top]

# Tentukan X & Y berdasarkan 2 bobot terbesar
sorted_idx = np.argsort(bobot)[::-1]

```

```

x_col = KRITERIA[sorted_idx[0]]
y_col = KRITERIA[sorted_idx[1]] if bobot.sum() > 0 else KRITERIA[1]

ax6.scatter(df_bot_scatter[x_col], df_bot_scatter[y_col],
            alpha=0.25, color='#b7e4c7', s=20, label='Lainnya')
scatter_top = ax6.scatter(df_top_scatter[x_col],
df_top_scatter[y_col],
                        alpha=0.85, c=df_top_scatter['Nilai V'],
                        cmap='YlGn', s=60, label='Top 10% Nilai
V', zorder=5,
                        edgecolors='#1b4332', linewidths=0.5)
plt.colorbar(scatter_top, ax=ax6, label='Nilai V (SAW)')
ax6.set_xlabel(f"{LABEL_MAP[x_col]} (bobot terbesar:
{bobot_pct[sorted_idx[0]]}%)", fontsize=11)
ax6.set_ylabel(f"{LABEL_MAP[y_col]} (bobot terbesar ke-2:
{bobot_pct[sorted_idx[1]]}%)", fontsize=11)
ax6.set_title(
    f"Sebaran Data: {LABEL_MAP[x_col]} vs {LABEL_MAP[y_col]}\n"
    f"Filter: {crop_filter} | Titik emas = Top 10% Nilai V",
    fontsize=12, fontweight='bold'
)
ax6.legend()
ax6.set_facecolor('#f8fffe')
ax6.grid(linestyle='--', alpha=0.4)
plt.tight_layout()
st.pyplot(fig6)
plt.close()

st.markdown("---")
st.info(
    "💡 **Tips:** Coba geser slider bobot di sidebar kiri dan
perhatikan bagaimana "
    "Grafik 1, 2, 4, 5, dan 6 langsung berubah! "
    "Ganti filter tanaman untuk melihat perubahan pada Grafik 1, 3,
4, 5, dan 6."
)

# HALAMAN 5 - PROFIL KELOMPOK
# -----
elif halaman == "👥 Profil Kelompok":
    st.markdown("<div class='title-header'><h1>👥 Profil
Kelompok</h1><p>Praktikum SCPK 2025/2026</p></div>",
unsafe_allow_html=True)

    col1, col2 = st.columns(2)
    with col1:

```

```

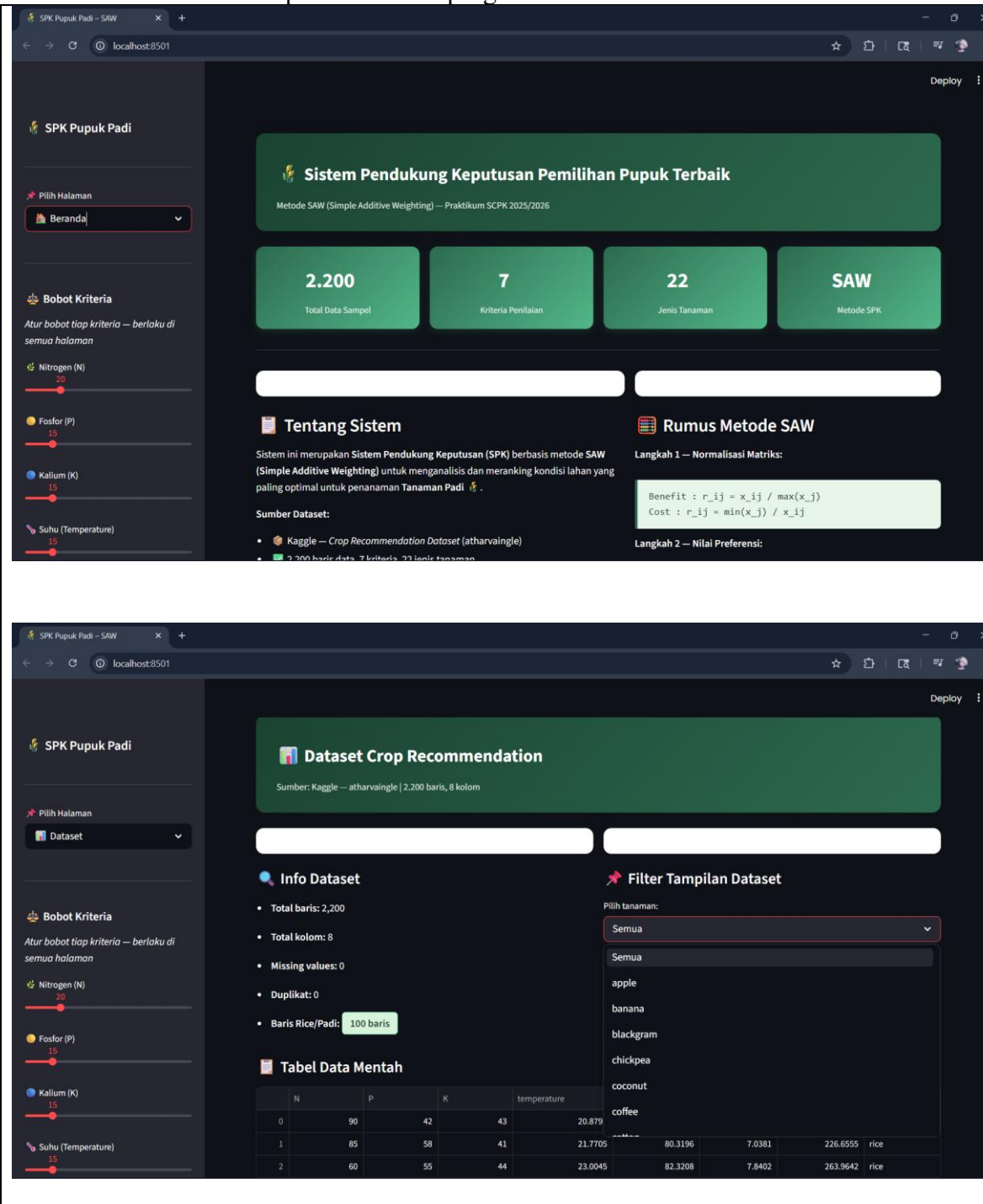
st.markdown("""
<div class='card' style='text-align:center'>
  <div style='font-size:4rem'>👤</div>
  <h2 style='color:#2d6a4f'>Muhammad Athalla Bagaskara</h2>
  <p style='font-size:1rem; color:#555'>NIM: 123240212</p>
  <hr>
  <p>Anggota Kelompok 1</p>
</div>
""", unsafe_allow_html=True)
with col2:
  st.markdown("""
<div class='card' style='text-align:center'>
  <div style='font-size:4rem'>👤</div>
  <h2 style='color:#2d6a4f'>Muhammad Ridho Hikmatul
Maulana</h2>
  <p style='font-size:1rem; color:#555'>NIM: 123240218</p>
  <hr>
  <p>Anggota Kelompok 2</p>
</div>
""", unsafe_allow_html=True)

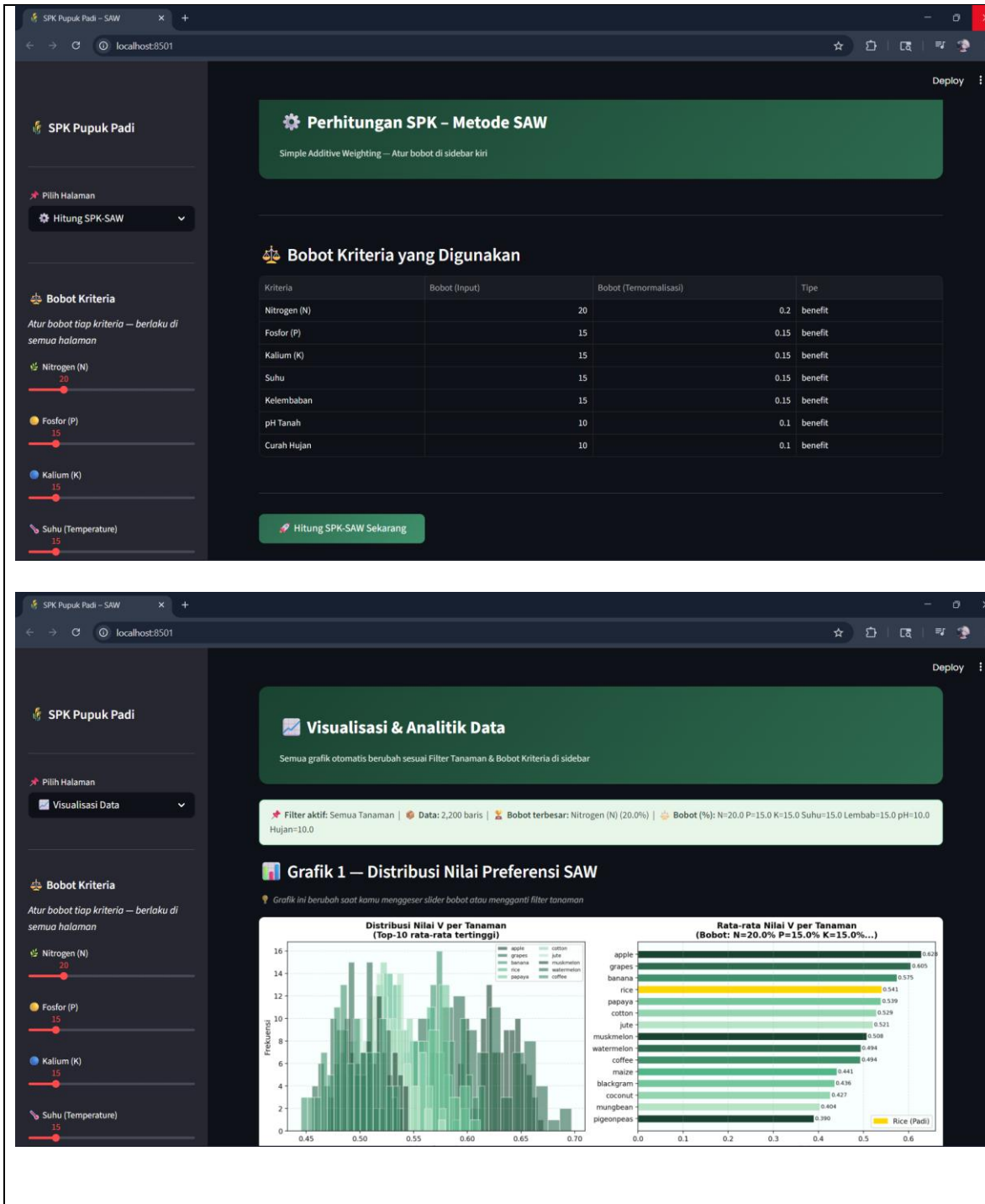
st.markdown("---")
st.markdown("""
<div class='card'>
  <h3 style='color:#2d6a4f'>📄 Detail Proyek Akhir</h3>
  <table style='width:100%; font-size:0.95rem'>
    <tr><td style='padding:6px;font-weight:600'>Judul</td><td>Sistem Pendukung Keputusan berbasis metode SAW
untuk menganalisis Pemilihan Pupuk Terbaik untuk Penanaman Tanaman
Padi</td></tr>
    <tr><td style='padding:6px;font-weight:600'>Metode</td><td>SAW (Simple Additive Weighting)</td></tr>
    <tr><td style='padding:6px;font-weight:600'>Dataset</td><td>Crop Recommendation Dataset – Kaggle
(atharvaingle), 2.200 baris</td></tr>
    <tr><td style='padding:6px;font-weight:600'>Kriteria</td><td>N, P, K, Temperature, Humidity, pH,
Rainfall (7 kriteria)</td></tr>
    <tr><td style='padding:6px;font-weight:600'>Tools</td><td>Python, Streamlit, Pandas, NumPy, Matplotlib,
Seaborn</td></tr>
    <tr><td style='padding:6px;font-weight:600'>Mata
Kuliah</td><td>Praktikum SCPK (Sistem Cerdas Pendukung Keputusan)
2025/2026</td></tr>
  </table>
</div>

```

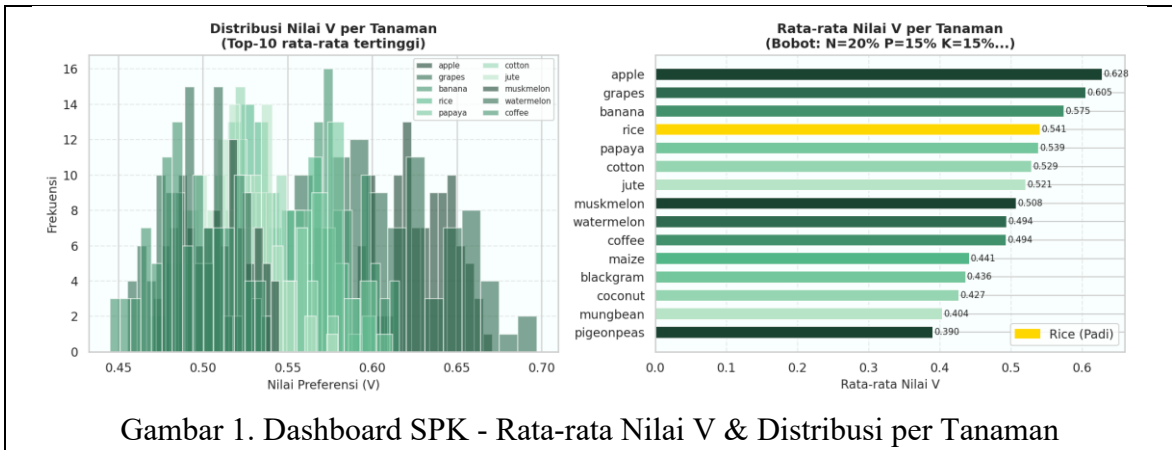
```
""", unsafe_allow_html=True)
```

Berikut adalah tampilan streamlit program

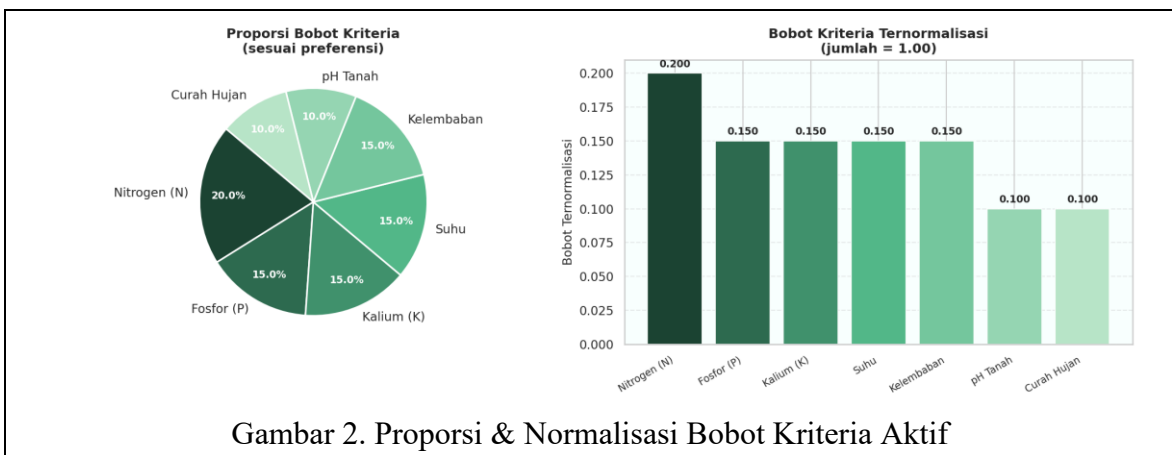




Berikut adalah visualisasi analitik dan grafik distribusi yang dihasilkan oleh sistem pendukung keputusan:

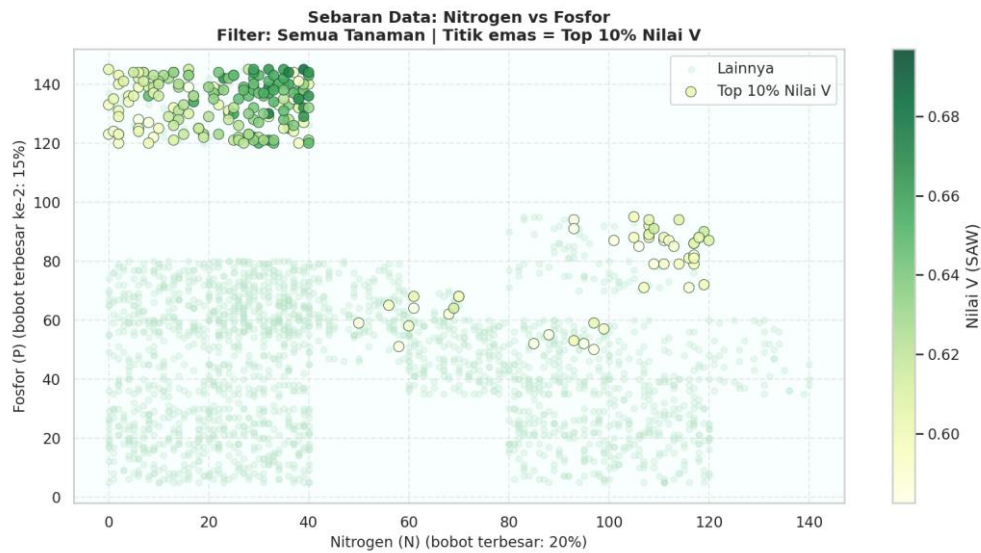


Gambar 1. Dashboard SPK - Rata-rata Nilai V & Distribusi per Tanaman

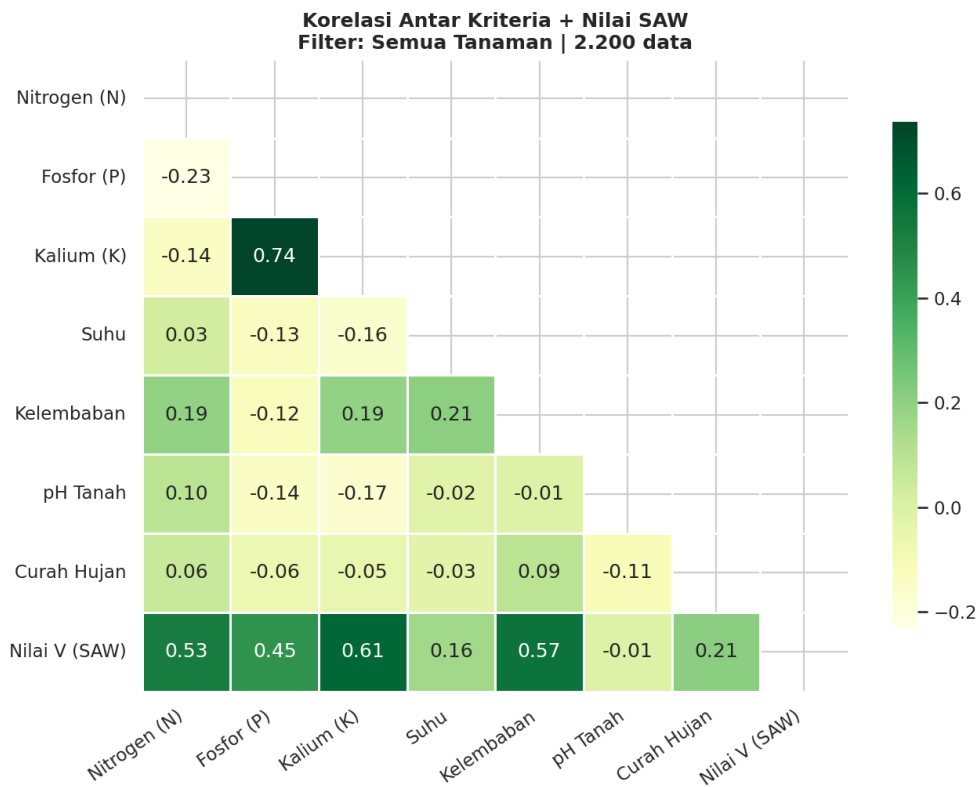


Gambar 2. Proporsi & Normalisasi Bobot Kriteria Aktif

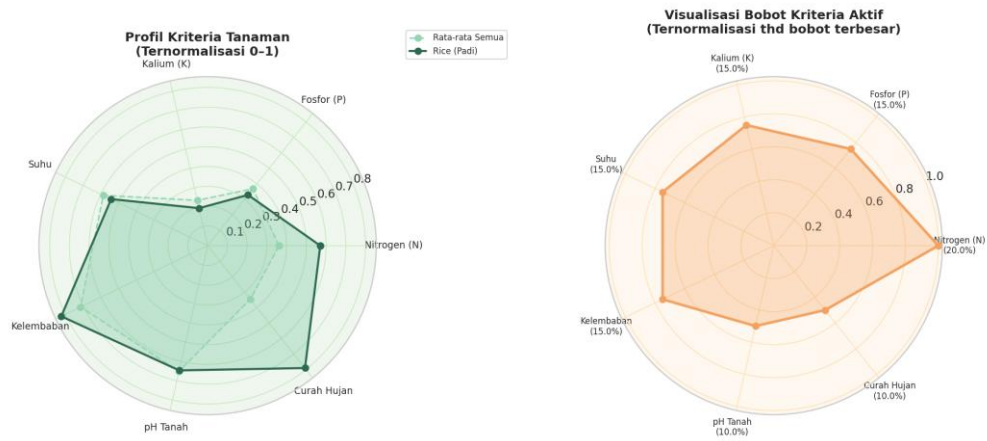
Gambar 6. Scatter Plot Sebaran Data: Nitrogen vs Fosfor



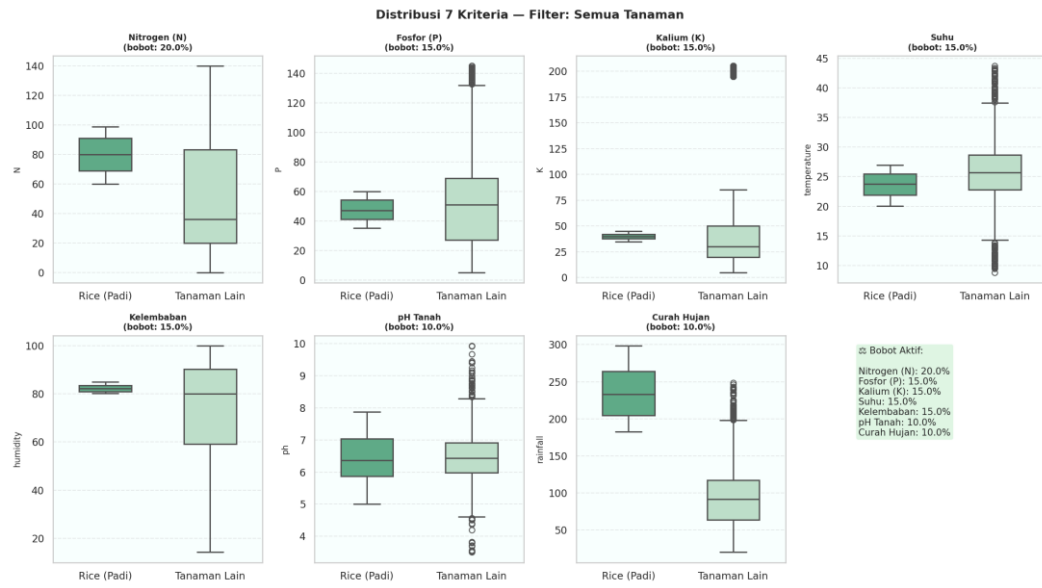
Gambar 5. Heatmap Korelasi Antara 7 Kriteria & Nilai Preferensi SAW



Gambar 4. Radar Chart Profil Kriteria Tanaman & Visualisasi Bobot Aktif



Gambar 3. Boxplot Distribusi 7 Kriteria Lahan (Rice vs Tanaman Lain)



BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026				
		Minggu				
		1	2	3	4	dst
1	Analisis Kebutuhan & Dataset	X				
2	Perancangan Model SPK SAW		X			
3	Coding GUI Streamlit & Grafik			X		
4	Pengujian Sistem & Pengumpulan Data				X	
dst	Penyusunan & Pengumpulan Laporan					X

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Muhammad Athalla Bagaskara	123240212	1. Pengumpulan dataset. 2. Implementasi metode SAW pada python, dan streamlit.
2	Muhammad Ridho Hikmatul Maulana	123240218	2. Implementasi modul grafik analitik menggunakan Matplotlib dan Seaborn (radar chart, heatmap, boxplot). 3. Penyusunan Laporan Proyek Akhir serta perapian dokumen.

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Telah berhasil dibangun aplikasi Sistem Pendukung Keputusan (SPK) berbasis web dengan Streamlit menggunakan metode Simple Additive Weighting (SAW) untuk menentukan rekomendasi kondisi lahan dan pemupukan NPK terbaik bagi tanaman padi. Melalui pemfilteran data padi dari Crop Recommendation Dataset, sistem mampu memproses 100 baris data secara akurat dan menghasilkan peringkat alternatif berdasarkan bobot dinamis yang diinputkan pengguna. Berdasarkan hasil pengujian dengan bobot preferensi Nitrogen (20%), Fosfor (15%), Kalium (15%), Suhu (15%), Kelembaban (15%), pH (10%), dan Curah Hujan (10%), alternatif terbaik memberikan nilai preferensi tertinggi yang menunjukkan kondisi hara dan lingkungan paling optimum untuk tanaman padi. Integrasi visualisasi grafik dinamis (seperti radar chart, boxplot kriteria, heatmap korelasi, dan histogram preferensi) berhasil mempermudah analisis komparatif antar alternatif hara tanah secara interaktif.

4.3 Saran

1. Pengembangan Dataset: Disarankan untuk menambahkan kriteria hara mikro (seperti C-Organik, Sulfur, Seng, Besi) serta alternatif pupuk spesifik merk dagang agar rekomendasi lebih praktis bagi petani.
2. Kombinasi Metode: Untuk pengembangan sistem di masa depan, bobot kriteria dinamis dapat dikombinasikan dengan metode pembobotan objektif seperti AHP (Analytic Hierarchy Process) atau Entropy agar penentuan bobot awal lebih ilmiah dan tidak subjektif.
3. Integrasi Sensor IoT: Menghubungkan aplikasi web ini dengan sensor IoT lapangan (soil moisture, pH meter digital) secara real-time agar rekomendasi pemupukan langsung menyesuaikan kondisi tanah saat itu juga.

DAFTAR PUSTAKA

- Atharvaingle. (2020). Crop Recommendation Dataset. Kaggle.
<https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>
- Fishburn, P. C. (1967). Additive Utilities with Incomplete Product Sets: Application to Decision Making. *Operations Research*, 15(3), 537-542.
- Nofriansyah, D. (2014). Konsep Data Mining Vs Sistem Pendukung Keputusan. Deepublish.
- Turban, E., Aronson, J. E., & Liang, T. P. (2005). *Decision Support Systems and Intelligent Systems*. Pearson Education.

Source Dataset:

<https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>